EX.NO: 11

MAP COLORING USING CONSTRAINT SATISFACTION PROBLEM (CSP)

AIM:

To write a Python program to solve the Map Coloring problem by implementing a Constraint Satisfaction Problem (CSP) using backtracking.

ALGORITHM:

Step 1: Define the variables (regions of the map), the domain (available colors) and the neighbor relation between regions.
Step 2: Choose an unassigned region.
Step 3: Try each color in the domain; a color is valid only if no neighboring region already has the same color.
Step 4: If the color is valid, assign it and recursively assign the next region.
Step 5: If no color works, backtrack: undo the last assignment and try the next color.
Step 6: Stop when all regions are colored and print the solution.

PROGRAM:

```
# Map coloring using Backtracking CSP (Australia map)
variables = ['WA', 'NT', 'SA', 'Q', 'NSW', 'V', 'T']
domains = ['Red', 'Green', 'Blue']

neighbors = {
    'WA': ['NT', 'SA'],
    'NT': ['WA', 'SA', 'Q'],
    'SA': ['WA', 'NT', 'Q', 'NSW', 'V'],
    'Q':  ['NT', 'SA', 'NSW'],
    'NSW': ['Q', 'SA', 'V'],
    'V':  ['SA', 'NSW'],
    'T':  []
}

def is_consistent(var, color, assignment):
    for n in neighbors[var]:
        if assignment.get(n) == color:
            return False
    return True

def backtrack(assignment):
    if len(assignment) == len(variables):
        return assignment
    var = next(v for v in variables if v not in assignment)
    for color in domains:
        if is_consistent(var, color, assignment):
            assignment[var] = color
            result = backtrack(assignment)
            if result:
                return result
            del assignment[var]
    return None

solution = backtrack({})
if solution:
    print("Map Coloring Solution:")
    for region, color in solution.items():
        print(f"  {region}: {color}")
else:
    print("No solution exists")
```

192321131

OUTPUT:

```
Map Coloring Solution:
  WA: Red
  NT: Green
  SA: Blue
  Q: Red
  NSW: Green
  V: Red
  T: Red
```

RESULT:

Thus the Python program for Map Coloring using CSP was written and executed successfully.

192321131

EX.NO: 12

TIC TAC TOE GAME

AIM:

To write a Python program to implement the two-player Tic Tac Toe game.

ALGORITHM:

Step 1: Create a 3x3 board initialized with empty cells.
Step 2: Players X and O take turns entering a row and column (1-3).
Step 3: Reject the move if it is outside the board or the cell is already occupied.
Step 4: After every move, check all rows, columns and both diagonals for three identical marks.
Step 5: If a player has three in a line, declare that player the winner.
Step 6: If the board is full and nobody has won, declare a draw; otherwise switch the player and repeat.

PROGRAM:

```
def print_board(board):
    print()
    for i in range(3):
        print(" " + " | ".join(board[i]))
        if i < 2:
            print("---+---+---")
    print()

def check_winner(board, player):
    for i in range(3):
        if all(board[i][j] == player for j in range(3)): return True
        if all(board[j][i] == player for j in range(3)): return True
    if all(board[i][i] == player for i in range(3)): return True
    if all(board[i][2 - i] == player for i in range(3)): return True
    return False

def is_full(board):
    return all(cell != ' ' for row in board for cell in row)

def play():
    board = [[' ']*3 for _ in range(3)]
    player = 'X'
    print("Tic Tac Toe! Enter row and column (1-3).")
    while True:
        print_board(board)
        try:
            r = int(input(f"Player {player} - row: ")) - 1
            c = int(input(f"Player {player} - col: ")) - 1
        except ValueError:
            print("Enter numbers only!")
            continue
        if not (0 <= r < 3 and 0 <= c < 3) or board[r][c] != ' ':
            print("Invalid move, try again.")
            continue
        board[r][c] = player
        if check_winner(board, player):
            print_board(board)
            print(f"Player {player} wins!")
            break
        if is_full(board):
            print_board(board)
            print("It's a draw!")
            break
        player = 'O' if player == 'X' else 'X'

play()
```

192321131

OUTPUT:

```
Tic Tac Toe! Enter row and column (1-3).

   |   |
---+---+---
   |   |
---+---+---
   |   |

Player X - row: 2
Player X - col: 2

   |   |
---+---+---
   | X |
---+---+---
   |   |

Player O - row: 1
Player O - col: 1

 O |   |
---+---+---
   | X |
---+---+---
   |   |

Player X - row: 3
Player X - col: 3

 O |   |
---+---+---
   | X |
---+---+---
   |   | X

Player O - row: 1
Player O - col: 3

 O |   | O
---+---+---
   | X |
---+---+---
   |   | X

Player X - row: 1
Player X - col: 2

 O | X | O
---+---+---
   | X |
---+---+---
   |   | X

Player O - row: 3
Player O - col: 1

 O | X | O
---+---+---
   | X |
---+---+---
 O |   | X

Player X - row: 3
Player X - col: 2

 O | X | O
---+---+---
   | X |
---+---+---
 O | X | X
Player X wins!
```

192321131

RESULT:

Thus the Python program for the Tic Tac Toe game was written and executed successfully.

192321131

EX.NO: 13

MINIMAX ALGORITHM FOR GAMING

AIM:

To write a Python program to implement the Minimax algorithm for game playing.

ALGORITHM:

Step 1: Represent the game tree as a binary tree whose leaf nodes hold the utility values.
Step 2: If a leaf node is reached, return its value.
Step 3: At a MAX node, return the maximum of the values of its children.
Step 4: At a MIN node, return the minimum of the values of its children.
Step 5: Start from the root as a MAX node and recursively evaluate the tree.
Step 6: Print the optimal value obtained at the root.

PROGRAM:

```
import math

# Minimax on a game tree: leaf values given
def minimax(depth, node_index, is_maximizing, scores, height):
    if depth == height:
        return scores[node_index]
    if is_maximizing:
        return max(minimax(depth + 1, node_index * 2, False, scores, height),
                   minimax(depth + 1, node_index * 2 + 1, False, scores, height))
    else:
        return min(minimax(depth + 1, node_index * 2, True, scores, height),
                   minimax(depth + 1, node_index * 2 + 1, True, scores, height))

scores = [3, 5, 2, 9, 12, 5, 23, 23]
height = int(math.log2(len(scores)))

print("Leaf values:", scores)
print("The optimal value is:", minimax(0, 0, True, scores, height))
```

OUTPUT:

```
Leaf values: [3, 5, 2, 9, 12, 5, 23, 23]
The optimal value is: 12
```

RESULT:

Thus the Python program for the Minimax algorithm was written and executed successfully.

192321131

EX.NO: 14

ALPHA-BETA PRUNING ALGORITHM FOR GAMING

AIM:

To write a Python program to implement the Alpha-Beta pruning algorithm for game playing.

ALGORITHM:

Step 1: Start at the root with alpha = -infinity and beta = +infinity.
Step 2: At a leaf node, return its utility value.
Step 3: At a MAX node, update best = max(best, child value) and alpha = max(alpha, best).
Step 4: At a MIN node, update best = min(best, child value) and beta = min(beta, best).
Step 5: If beta <= alpha, prune (stop exploring) the remaining children of that node.
Step 6: Return the value at the root as the optimal value.

PROGRAM:

```
import math

def alpha_beta(depth, node_index, is_maximizing, scores, height, alpha, beta):
    if depth == height:
        return scores[node_index]

    if is_maximizing:
        best = -math.inf
        for i in range(2):
            val = alpha_beta(depth + 1, node_index * 2 + i, False,
                             scores, height, alpha, beta)
            best = max(best, val)
            alpha = max(alpha, best)
            if beta <= alpha:
                print(f"Pruned at depth {depth}, node {node_index} (MAX)")
                break
        return best
    else:
        best = math.inf
        for i in range(2):
            val = alpha_beta(depth + 1, node_index * 2 + i, True,
                             scores, height, alpha, beta)
            best = min(best, val)
            beta = min(beta, best)
            if beta <= alpha:
                print(f"Pruned at depth {depth}, node {node_index} (MIN)")
                break
        return best

scores = [3, 5, 6, 9, 1, 2, 0, -1]
height = int(math.log2(len(scores)))

print("Leaf values:", scores)
result = alpha_beta(0, 0, True, scores, height, -math.inf, math.inf)
print("The optimal value is:", result)
```

OUTPUT:

```
Leaf values: [3, 5, 6, 9, 1, 2, 0, -1]
Pruned at depth 2, node 1 (MAX)
Pruned at depth 1, node 1 (MIN)
The optimal value is: 5
```

RESULT:

Thus the Python program for Alpha-Beta pruning was written and executed successfully.

192321131

EX.NO: 15

DECISION TREE

AIM:

To write a Python program to implement a Decision Tree classifier using the Iris dataset.

ALGORITHM:

Step 1: Import the required libraries and load the Iris dataset.
Step 2: Split the dataset into training (70%) and testing (30%) sets.
Step 3: Create a DecisionTreeClassifier using entropy (information gain) as the criterion.
Step 4: Train the model using the training data.
Step 5: Predict the test data and compute the accuracy.
Step 6: Display the tree rules and predict the class of a new sample.

PROGRAM:

```
# pip install scikit-learn
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Load dataset
iris = load_iris()
X, y = iris.data, iris.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)

# Train model
model = DecisionTreeClassifier(criterion='entropy', max_depth=3, random_state=42)
model.fit(X_train, y_train)

# Evaluate
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))

# Show tree rules
print("\nDecision Tree Rules:")
print(export_text(model, feature_names=iris.feature_names))

# Predict a new sample
sample = [[5.1, 3.5, 1.4, 0.2]]
print("Prediction for sample:", iris.target_names[model.predict(sample)[0]])
```

192321131

OUTPUT:

```
Accuracy: 0.9777777777777777

Decision Tree Rules:
|--- petal length (cm) <= 2.45
|   |--- class: 0
|--- petal length (cm) >  2.45
|   |--- petal length (cm) <= 4.75
|   |   |--- petal width (cm) <= 1.60
|   |   |   |--- class: 1
|   |   |--- petal width (cm) >  1.60
|   |   |   |--- class: 2
|   |--- petal length (cm) >  4.75
|   |   |--- petal length (cm) <= 5.15
|   |   |   |--- class: 2
|   |   |--- petal length (cm) >  5.15
|   |   |   |--- class: 2

Prediction for sample: setosa
```

RESULT:

Thus the Python program to implement a Decision Tree was written and executed successfully.

192321131

EX.NO: 16

FEED FORWARD NEURAL NETWORK

AIM:

To write a Python program to implement a Feed Forward Neural Network with backpropagation to learn the XOR function.

ALGORITHM:

Step 1: Define the sigmoid activation function and its derivative.
Step 2: Prepare the XOR input data and target outputs.
Step 3: Initialize the weights randomly and the biases to zero (2 input, 4 hidden, 1 output neurons).
Step 4: Forward pass: compute the hidden layer and output layer activations.
Step 5: Compute the error and backpropagate it to obtain the gradients of both layers.
Step 6: Update the weights and biases using the learning rate; repeat for the given number of epochs.
Step 7: Print the final predictions.

PROGRAM:

```
import numpy as np

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)

# XOR dataset
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([[0], [1], [1], [0]])

np.random.seed(42)
input_size, hidden_size, output_size = 2, 4, 1

W1 = np.random.uniform(size=(input_size, hidden_size))
b1 = np.zeros((1, hidden_size))
W2 = np.random.uniform(size=(hidden_size, output_size))
b2 = np.zeros((1, output_size))

lr = 0.5
epochs = 10000

for epoch in range(epochs):
    # Forward pass
    hidden = sigmoid(np.dot(X, W1) + b1)
    output = sigmoid(np.dot(hidden, W2) + b2)

    # Backpropagation
    error = y - output
    d_output = error * sigmoid_derivative(output)
    d_hidden = d_output.dot(W2.T) * sigmoid_derivative(hidden)

    # Update weights
    W2 += hidden.T.dot(d_output) * lr
    b2 += np.sum(d_output, axis=0, keepdims=True) * lr
    W1 += X.T.dot(d_hidden) * lr
    b1 += np.sum(d_hidden, axis=0, keepdims=True) * lr

    if epoch % 2000 == 0:
        print(f"Epoch {epoch}, Loss: {np.mean(np.abs(error)):.4f}")

print("\nFinal Predictions:")
for inp, out in zip(X, output):
    print(f"Input: {inp} -> Output: {out[0]:.3f} -> Rounded: {int(round(out[0]))}")
```

192321131

OUTPUT:

```
Epoch 0, Loss: 0.4965
Epoch 2000, Loss: 0.0506
Epoch 4000, Loss: 0.0295
Epoch 6000, Loss: 0.0225
Epoch 8000, Loss: 0.0188

Final Predictions:
Input: [0 0] -> Output: 0.019 -> Rounded: 0
Input: [0 1] -> Output: 0.984 -> Rounded: 1
Input: [1 0] -> Output: 0.984 -> Rounded: 1
Input: [1 1] -> Output: 0.015 -> Rounded: 0
```

RESULT:

Thus the Python program for a Feed Forward Neural Network was written and executed successfully.

192321131